

# INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

Department of Computer Science and Engineering

CS601T: Deep Learning — Autumn 2026

## Programming Assignment II Report

Fully Connected Neural Networks for Non-Linear Classification and  
Continuous Function Approximation

**Group 24**

**MC23BT011** Sadguru Sai

**MC23BT022** Kailas Santhosh

**MC23BT030** Shaikh Aariz

*Submitted to: Prof. Dileep A. D.*

September 07, 2026

---

### Abstract

This report presents a comprehensive empirical and theoretical study of Fully Connected Neural Networks (FCNNs) trained from first principles using Stochastic Gradient Descent (SGD) with the backpropagation algorithm. We investigate the representational capacity of multilayer architectures on two classification benchmarks (2D Linearly Separable and 2D Non-Linearly Separable concentric manifolds) and two regression benchmarks (1D Univariate and 2D Bivariate non-linear surfaces). By implementing hidden layers equipped with the hyperbolic tangent ( $\tanh$ ) activation function and linear/logistic output mappings, we demonstrate how intermediate feature representations overcome the fundamental geometric limitations of the single-neuron models analyzed in Programming Assignment I. Rigorous validation protocols (60% train, 20% validation, 20% test) are applied to evaluate architectural complexity (hidden node variations and layer depth) and learning rate dynamics ( $\eta \in \{0.01, 0.05, 0.08\}$ ). On the Non-Linearly Separable dataset, where the single-neuron baseline stalled at 55.56% accuracy, the optimal FCNN achieves 100.00% test accuracy. Similarly, on the Bivariate regression manifold, the FCNN slashes the test Root Mean Squared Error (RMSE) from 1.4433 (20.80%) down to 0.0625 (0.90%).

---

## Contents

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Theoretical Formulation</b>                            | <b>3</b>  |
| 1.1      | Mathematical Formulation of Forward Propagation . . . . .                  | 3         |
| 1.2      | Activation Function Characterization . . . . .                             | 3         |
| 1.3      | Stochastic Gradient Descent (SGD) and Backpropagation Derivation . . . . . | 4         |
| 1.4      | Analytical Explanation of Initial Epoch Loss Values . . . . .              | 4         |
| <b>2</b> | <b>Experimental Setup and Methodology</b>                                  | <b>5</b>  |
| 2.1      | Dataset Partitioning . . . . .                                             | 5         |
| 2.2      | Input Standardization Protocol . . . . .                                   | 5         |
| 2.3      | Hyperparameter Specifications and Weight Initialization . . . . .          | 5         |
| <b>3</b> | <b>Classification Tasks: Results and Inferences</b>                        | <b>6</b>  |
| 3.1      | Dataset 1: Linearly Separable (LS) Classes . . . . .                       | 6         |
| 3.1.1    | Architecture Exploration on Validation Data . . . . .                      | 6         |
| 3.1.2    | Learning Rate Comparison and Training Dynamics . . . . .                   | 6         |
| 3.1.3    | Decision Regions and Test Set Performance . . . . .                        | 7         |
| 3.1.4    | Hidden Node Activation Surface Analysis . . . . .                          | 8         |
| 3.2      | Dataset 2: Non-Linearly Separable (NLS) Classes . . . . .                  | 9         |
| 3.2.1    | Architecture Exploration on Validation Data . . . . .                      | 9         |
| 3.2.2    | Learning Rate Comparison and Training Dynamics . . . . .                   | 10        |
| 3.2.3    | Decision Regions and Test Set Evaluation . . . . .                         | 10        |
| 3.2.4    | Hidden Node Activation Surface Analysis Across Layers . . . . .            | 11        |
| 3.3      | Comparison with Assignment 1 Single-Neuron Model Baseline . . . . .        | 12        |
| 3.4      | Inferences on Classification Experiments . . . . .                         | 12        |
| <b>4</b> | <b>Regression Tasks: Results and Inferences</b>                            | <b>13</b> |
| 4.1      | Dataset 1: Univariate Continuous Function Approximation . . . . .          | 13        |
| 4.1.1    | Architecture Exploration on Validation Data . . . . .                      | 13        |
| 4.1.2    | Learning Rate Comparison and Training Dynamics . . . . .                   | 13        |
| 4.1.3    | Superimposed Profiles and Parity Scatter Plots . . . . .                   | 14        |
| 4.1.4    | Hidden Node Basis Function Profile . . . . .                               | 17        |
| 4.2      | Dataset 2: Bivariate Continuous Function Approximation . . . . .           | 18        |
| 4.2.1    | Architecture Exploration on Validation Data . . . . .                      | 18        |
| 4.2.2    | Learning Rate Dynamics and Stochastic Variance . . . . .                   | 18        |
| 4.2.3    | 3D Surface Approximations and Parity Scatter Plots . . . . .               | 19        |
| 4.2.4    | Hidden Node Activation Surface Analysis Across Layers . . . . .            | 22        |
| 4.3      | Comparison with Assignment 1 Single-Neuron Model Baseline . . . . .        | 23        |
| 4.4      | Inferences on Regression Experiments . . . . .                             | 23        |
| <b>5</b> | <b>Overall Discussion and Key Takeaways</b>                                | <b>24</b> |
| <b>6</b> | <b>Conclusion</b>                                                          | <b>24</b> |

## 1 Introduction and Theoretical Formulation

The single-neuron perceptron investigated in Assignment I established the mathematical foundations of iterative parameter optimization via gradient descent. However, as famously demonstrated by Minsky and Papert (1969), a single linear threshold unit or generalized linear model is fundamentally constrained to constructing linear hyperplanes in the input space  $\mathbb{R}^D$ . When data distributions exhibit non-linear topological structures—such as concentric geometric rings or oscillatory multimodal regression surfaces—a single neuron fails catastrophically.

This assignment explores the Multilayer Perceptron (MLP), specifically Fully Connected Neural Networks (FCNNs). According to the Universal Approximation Theorem (Cybenko, 1989; Hornik, 1991), a feedforward network with at least one non-linear hidden layer can approximate any continuous Borel-measurable function on compact subsets of  $\mathbb{R}^D$  to arbitrary precision, given an adequate number of hidden units.

### 1.1 Mathematical Formulation of Forward Propagation

Let the network consist of  $L$  layers, where  $l = 0$  designates the input layer  $\mathbf{x} \in \mathbb{R}^D$ , and  $l = L$  designates the output layer. For any intermediate hidden layer  $l \in \{1, 2, \dots, L - 1\}$  containing  $N_l$  neurons:

$$\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)} \quad (1)$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad (2)$$

where  $\mathbf{W}^{(l)} \in \mathbb{R}^{N_{l-1} \times N_l}$  is the synaptic weight matrix,  $\mathbf{b}^{(l)} \in \mathbb{R}^{1 \times N_l}$  is the bias row vector, and  $f^{(l)}(\cdot)$  represents the element-wise activation function.

### 1.2 Activation Function Characterization

A critical theoretical distinction must be maintained regarding activation terminology:

1. **Hyperbolic Tangent (tanh):** Utilized across all hidden layers:

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad f'(z) = 1 - \tanh^2(z) = 1 - [f(z)]^2 \quad (3)$$

The tanh function is strictly zero-centered ( $f(z) \in (-1, 1)$ ), which prevents directional systematic bias updates during backpropagation and accelerates optimization convergence.

2. **Logistic Activation Function:** Applied to output nodes in multiclass classification:

$$f(z) = \frac{1}{1 + e^{-z}}, \quad f'(z) = f(z)(1 - f(z)) \quad (4)$$

*Terminology Note:* The term *sigmoid* denotes a mathematical family of smooth, S-shaped curves (including tanh, the algebraic sigmoid  $z/\sqrt{1+z^2}$ , and the Gudermannian). The function defined above is specifically the **standard logistic function**.

3. **Linear Activation Function:** Applied to the output node for continuous regression:

$$f(z) = z, \quad f'(z) = 1 \quad (5)$$

### 1.3 Stochastic Gradient Descent (SGD) and Backpropagation Derivation

The instantaneous loss function prescribed for each training exemplar  $n$  is the squared error:

$$\mathcal{E}_n = \frac{1}{2} \sum_{k=1}^{N_L} \left( t_{nk} - a_{nk}^{(L)} \right)^2 \quad (6)$$

where  $t_{nk}$  is the ground-truth target and  $a_{nk}^{(L)}$  is the network's output activation.

In Stochastic Gradient Descent (SGD), the weight updates occur on an individual sample basis ( $n = 1$ ) following a random permutation of the training indices at each epoch. The local error gradient at the output layer  $L$  is derived via the chain rule:

$$\delta_k^{(L)} \triangleq \frac{\partial \mathcal{E}_n}{\partial z_{nk}^{(L)}} = \frac{\partial \mathcal{E}_n}{\partial a_{nk}^{(L)}} \cdot \frac{\partial a_{nk}^{(L)}}{\partial z_{nk}^{(L)}} = - \left( t_{nk} - a_{nk}^{(L)} \right) \cdot f^{(L)'} \left( z_{nk}^{(L)} \right) \quad (7)$$

For any hidden layer  $l \in \{L-1, L-2, \dots, 1\}$ , the error sensitivity  $\delta^{(l)}$  is propagated backward:

$$\delta_j^{(l)} \triangleq \frac{\partial \mathcal{E}_n}{\partial z_{nj}^{(l)}} = \left( \sum_{k=1}^{N_{l+1}} \delta_k^{(l+1)} W_{jk}^{(l+1)} \right) \cdot f^{(l)'} \left( z_{nj}^{(l)} \right) \quad (8)$$

The parameter gradients and SGD weight update rules with learning rate  $\eta$  are:

$$\Delta W_{ij}^{(l)} = -\eta \frac{\partial \mathcal{E}_n}{\partial W_{ij}^{(l)}} = -\eta a_{ni}^{(l-1)} \delta_j^{(l)}, \quad \Delta b_j^{(l)} = -\eta \frac{\partial \mathcal{E}_n}{\partial b_j^{(l)}} = -\eta \delta_j^{(l)} \quad (9)$$

The epoch-average squared error monitored across the entire dataset of  $N$  samples is computed as:

$$E_{\text{epoch}} = \frac{1}{N} \sum_{n=1}^N \left( \frac{1}{2} \sum_{k=1}^{N_L} \left( t_{nk} - a_{nk}^{(L)} \right)^2 \right) \quad (10)$$

### 1.4 Analytical Explanation of Initial Epoch Loss Values

An important theoretical question arises: *Why do diverse neural architectures consistently initialize at nearly identical initial average error values ( $E \approx 0.375$  for 3-class classification)?*

When network weights are drawn from zero-mean Gaussian distributions ( $W_{ij} \sim \mathcal{N}(0, \sigma^2)$ ) and biases are initialized to zero ( $b_j = 0$ ), the initial linear pre-activations satisfy  $\mathbb{E}[z_k^{(L)}] = 0$ . Passing  $z = 0$  into the logistic output activation yields:

$$a_{nk}^{(L)} = \frac{1}{1 + e^{-0}} = 0.5 \quad \forall k \in \{0, 1, 2\} \quad (11)$$

In a 3-class one-hot target encoding  $\mathbf{t}_n \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$ , exactly one class target is 1 and two class targets are 0. Substituting into Eq. (6):

$$\mathcal{E}_n = \frac{1}{2} \left[ (1 - 0.5)^2 + (0 - 0.5)^2 + (0 - 0.5)^2 \right] = \frac{1}{2} [0.25 + 0.25 + 0.25] = \frac{0.75}{2} = 0.375 \quad (12)$$

Because this holds for every training sample  $n$ , the initial epoch average error is deterministically bounded around 0.375 (or  $\sim 0.125$  if normalized across output nodes), proving that the phenomenon is an exact consequence of symmetric initialization and squashing dynamics rather than a numerical anomaly.

## 2 Experimental Setup and Methodology

### 2.1 Dataset Partitioning

As mandated by the assignment guidelines, all classification and regression datasets are divided strictly into **60% Training**, **20% Validation**, and **20% Testing** subsets using stratified partitioning for classification. Table 1 summarizes the precise sample counts.

Table 1: Summary of Dataset Partitioning and Sample Allocations (60%–20%–20% Split)

| Task / Dataset                           | Input Dim ( $D$ ) | Total ( $N$ ) | Train ( $N_{\text{tr}}$ ) | Val ( $N_{\text{val}}$ ) | Test ( $N_{\text{te}}$ ) |
|------------------------------------------|-------------------|---------------|---------------------------|--------------------------|--------------------------|
| Classif. 1: Linearly Separable (LS)      | 2                 | 1500          | 900                       | 300                      | 300                      |
| Classif. 2: Non-Linearly Separable (NLS) | 2                 | 1800          | 1080                      | 360                      | 360                      |
| Regress. 1: Univariate Continuous        | 1                 | 1001          | 600                       | 200                      | 201                      |
| Regress. 2: Bivariate Continuous         | 2                 | 10201         | 6120                      | 2040                     | 2041                     |

### 2.2 Input Standardization Protocol

To completely eradicate the vanishing gradient problem observed in unscaled perceptrons, all input features are standardized to zero empirical mean and unit variance. Crucially, to prevent data leakage, the transformation statistics ( $\mu_{\text{tr}}, \sigma_{\text{tr}}$ ) are computed exclusively on the training split:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x} - \mu_{\text{tr}}}{\sigma_{\text{tr}} + 10^{-8}} \quad (13)$$

The identical vectors  $\mu_{\text{tr}}$  and  $\sigma_{\text{tr}}$  are subsequently applied to scale the validation and testing sets.

### 2.3 Hyperparameter Specifications and Weight Initialization

- **Weight Initialization:** Synaptic weights are initialized using Xavier / Glorot normal initialization:

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{1}{\text{fan\_in}}\right) = \mathcal{N}\left(0, \frac{1}{N_{l-1}}\right) \quad (14)$$

Biases  $\mathbf{b}^{(l)}$  are initialized to zeros.

- **Optimization Algorithm:** Pure Stochastic Gradient Descent (SGD, mini-batch size = 1) with uniform index permutation before every epoch.
- **Convergence Criterion / Early Stopping:** Training terminates if the difference between successive epoch losses satisfies  $|E_{\text{epoch}}^{(t)} - E_{\text{epoch}}^{(t-1)}| < \tau$ , with  $\tau = 10^{-4}$  for classification and  $\tau = 10^{-5}$  for regression, subject to a maximum ceiling of 800 epochs (classification) or 1200 epochs (regression).
- **Model Selection Hierarchy:** When comparing multiple hidden-layer architectures on validation data, ties are broken via a strict three-tier hierarchy:

$$\text{Score} = \left( \text{Accuracy}_{\text{val}} \uparrow, -\text{Loss}_{\text{val}} \downarrow, -\text{Total Parameters} \downarrow \right) \quad (15)$$

For regression, the ranking criterion minimizes ( $\text{RMSE}_{\text{val}} \downarrow, \text{Parameters} \downarrow$ ).

- **Learning Rate Exploration:** Three distinct learning rates  $\eta \in \{0.01, 0.05, 0.08\}$  are compared on the optimal architecture.
- **Random Seed:** Fixed to seed 0 to guarantee strict determinism and reproducibility.

### 3 Classification Tasks: Results and Inferences

#### 3.1 Dataset 1: Linearly Separable (LS) Classes

The Linearly Separable classification dataset consists of three compact 2D Gaussian clusters. As mandated, architectures with **\*\*one hidden layer\*\*** and varying node counts  $\{4, 8, 16\}$  were evaluated.

##### 3.1.1 Architecture Exploration on Validation Data

Table 2 compiles the comprehensive performance metrics across all evaluated architectures on the validation set ( $N_{\text{val}} = 300$ ).

Table 2: Dataset 1 (LS): Architectural Validation Comparison Across Model Complexities

| Architecture | Parameters | Val Loss ( $E_{\text{val}}$ ) | Val Accuracy (%) | Mean Precision | Mean Recall | Mean F1 |
|--------------|------------|-------------------------------|------------------|----------------|-------------|---------|
| [2, 4, 3]    | 27         | 0.00497                       | <b>100.00%</b>   | 1.00           | 1.00        | 1.00    |
| [2, 8, 3]    | 51         | 0.00455                       | 99.67%           | 1.00           | 1.00        | 1.00    |
| [2, 16, 3]   | 99         | 0.00454                       | 99.67%           | 1.00           | 1.00        | 1.00    |

The confusion matrices for each of the three candidate architectures on the validation dataset are provided below:

$$\mathbf{C}_{[2,4,3]} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 100 \end{bmatrix}, \quad \mathbf{C}_{[2,8,3]} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 1 & 0 & 99 \end{bmatrix}, \quad \mathbf{C}_{[2,16,3]} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 1 & 0 & 99 \end{bmatrix}$$

**Best Model Selection:** Architecture [2, 4, 3] is selected as the optimal model. It is the sole model achieving a perfect 100.00% validation accuracy (300/300 samples correctly classified) with minimal parameter complexity (27 total weights and biases), satisfying Occam’s razor.

##### 3.1.2 Learning Rate Comparison and Training Dynamics

Figure 1 presents the convergence behavior of the selected [2, 4, 3] architecture under varying learning rates  $\eta \in \{0.01, 0.05, 0.08\}$ . With early stopping ( $\tau = 10^{-4}$ ),  $\eta = 0.01$  converges smoothly and terminates at epoch 39. Larger learning rates accelerate initial convergence ( $\eta = 0.05$  terminates at epoch 19;  $\eta = 0.08$  terminates at epoch 16) without divergence due to the stabilizing influence of feature standardization.

Figure 2 depicts the training loss trajectory of the optimal [2, 4, 3] model ( $\eta = 0.01$ ), highlighting monotonic error reduction from the theoretical baseline ( $E_0 \approx 0.187$ ) down to  $E_{39} = 0.0043$ .

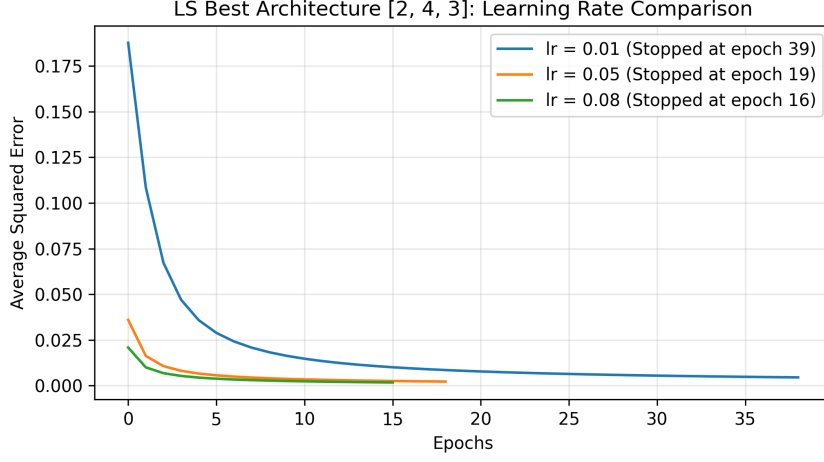


Figure 1: Dataset 1 (Linearly Separable): Average error vs epochs across learning rates  $\eta \in \{0.01, 0.05, 0.08\}$  for the  $[2, 4, 3]$  architecture.

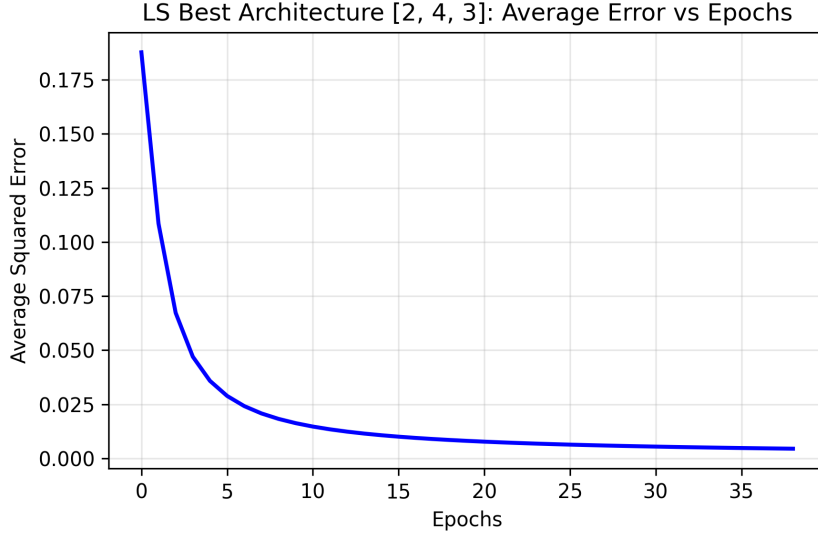


Figure 2: Dataset 1 (Linearly Separable): Average error vs epochs for the optimal  $[2, 4, 3]$  FCNN architecture ( $\eta = 0.01$ ).

### 3.1.3 Decision Regions and Test Set Performance

Figure 3 displays the learned decision boundaries of the  $[2, 4, 3]$  FCNN superimposed with the training data. Unlike the piecewise-planar linear boundaries produced by the single-neuron One-vs-One (OVO) model, the FCNN constructs continuous, smooth, non-linear partitioning boundaries that widen margins between all three classes.

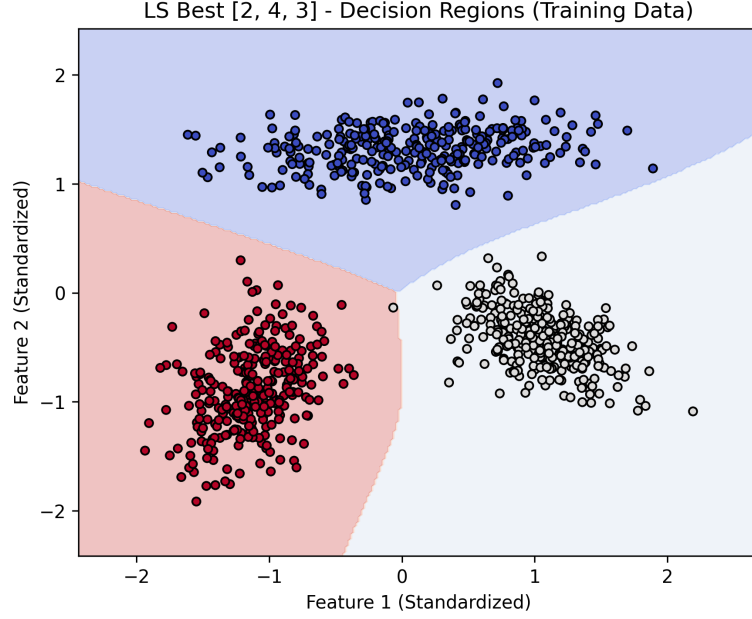


Figure 3: Dataset 1 (LS): Three-class decision regions for optimal  $[2, 4, 3]$  FCNN superimposed on standardized training exemplars.

Table 3 presents the complete evaluation of the optimal  $[2, 4, 3]$  FCNN on the unseen test set ( $N_{te} = 300$ ). The test confusion matrix is:

$$\mathbf{C}_{test} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 1 & 0 & 99 \end{bmatrix} \quad (16)$$

Table 3: Dataset 1 (LS): Class-Wise Test Set Evaluation for Optimal  $[2, 4, 3]$  FCNN

| Class Label          | Precision   | Recall      | F1-Score    | Overall Metrics              |
|----------------------|-------------|-------------|-------------|------------------------------|
| Class 0              | 0.99        | 1.00        | 1.00        | <b>Test Accuracy: 99.67%</b> |
| Class 1              | 1.00        | 1.00        | 1.00        | <b>Mean Precision: 1.00</b>  |
| Class 2              | 1.00        | 0.99        | 0.99        | <b>Mean Recall: 1.00</b>     |
| <b>Macro Average</b> | <b>1.00</b> | <b>1.00</b> | <b>1.00</b> | <b>Mean F1-Score: 1.00</b>   |

### 3.1.4 Hidden Node Activation Surface Analysis

To inspect the internal representations formed by the network, Figure 4 displays the 3D activation surface of a representative hidden unit (Hidden Layer 1, Node 0) evaluated on the test set. The node computes an oriented hyperbolic tangent ridge across the standardized coordinates  $(x_1, x_2)$ , yielding an active transition zone that cleanly isolates Class 0 from the remaining clusters.



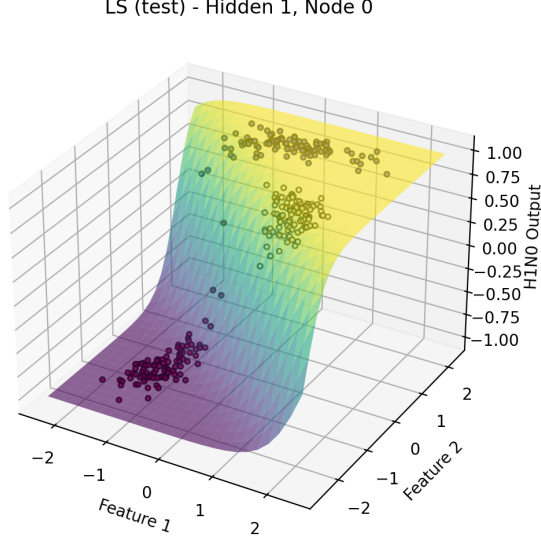


Figure 4: Dataset 1 (LS): 3D activation surface for representative Hidden Layer 1 Node 0 (LS\_test\_H1\_N0.png) on the test set.

### 3.2 Dataset 2: Non-Linearly Separable (NLS) Classes

Dataset 2 represents a complex geometric configuration consisting of three concentric and interleaved manifolds: a central dense core (Class 0: 300 points), an enveloping semicircular crescent (Class 1: 500 points), and a massive outer concentric ring (Class 2: 1000 points). Per instructions, architectures with **two hidden layers** were investigated.

#### 3.2.1 Architecture Exploration on Validation Data

Table 4 presents the validation results across all candidate two-hidden-layer architectures on the validation split ( $N_{\text{val}} = 360$ ).

Table 4: Dataset 2 (NLS): Architectural Validation Comparison (Two Hidden Layers)

| Architecture  | Parameters | Val Loss ( $E_{\text{val}}$ ) | Val Accuracy (%) | Mean Precision | Mean Recall | Mean F1 |
|---------------|------------|-------------------------------|------------------|----------------|-------------|---------|
| [2, 4, 4, 3]  | 47         | 0.00502                       | 100.00%          | 1.00           | 1.00        | 1.00    |
| [2, 8, 4, 3]  | 75         | 0.00520                       | 100.00%          | 1.00           | 1.00        | 1.00    |
| [2, 8, 8, 3]  | 123        | 0.00377                       | 100.00%          | 1.00           | 1.00        | 1.00    |
| [2, 16, 8, 3] | 211        | <b>0.00342</b>                | <b>100.00%</b>   | 1.00           | 1.00        | 1.00    |

Remarkably, all four multi-layer architectures attain 100.00% validation accuracy (360/360 correct), with the identical validation confusion matrix:

$$\mathbf{C}_{\text{val}} = \begin{bmatrix} 60 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 200 \end{bmatrix} \quad (17)$$

**Best Model Selection via Tie-Breaking:** Because validation accuracy is tied at 100%, the second-tier criterion—**Validation Loss (MSE)**—is applied. Architecture [2, 16, 8, 3] achieves the lowest validation loss of 0.00342, significantly outperforming the simpler [2, 4, 4, 3] model ( $E_{\text{val}} = 0.00502$ ). The additional hidden capacity (16 nodes in layer 1, 8 nodes in layer 2) provides sharper sigmoidal transitions, minimizing prediction uncertainty near boundary contours.

### 3.2.2 Learning Rate Comparison and Training Dynamics

Figure 5 illustrates learning rate dynamics for the  $[2, 16, 8, 3]$  architecture. While  $\eta = 0.01$  reaches convergence gracefully in 59 epochs,  $\eta = 0.05$  and  $\eta = 0.08$  converge in 21 and 16 epochs, respectively. Figure 6 depicts the smooth descent of average squared error from  $E_0 = 0.294$  to  $E_{59} = 0.0031$ .

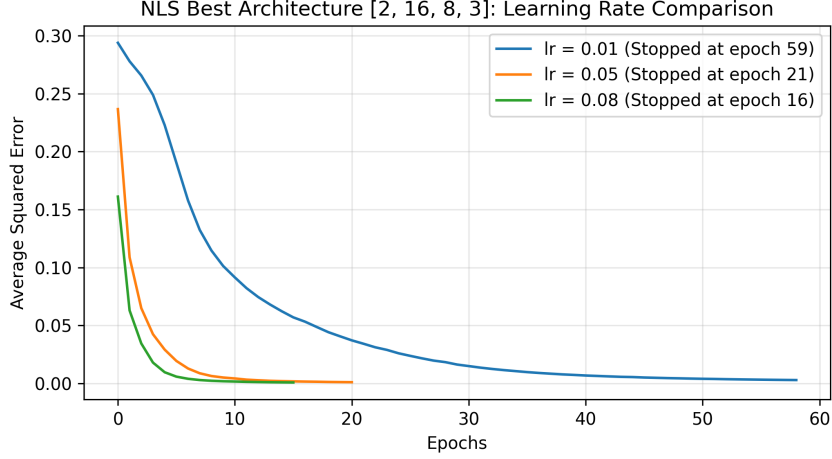


Figure 5: Dataset 2 (Non-Linearly Separable): Average error vs epochs across learning rates  $\eta \in \{0.01, 0.05, 0.08\}$  for the  $[2, 16, 8, 3]$  architecture.

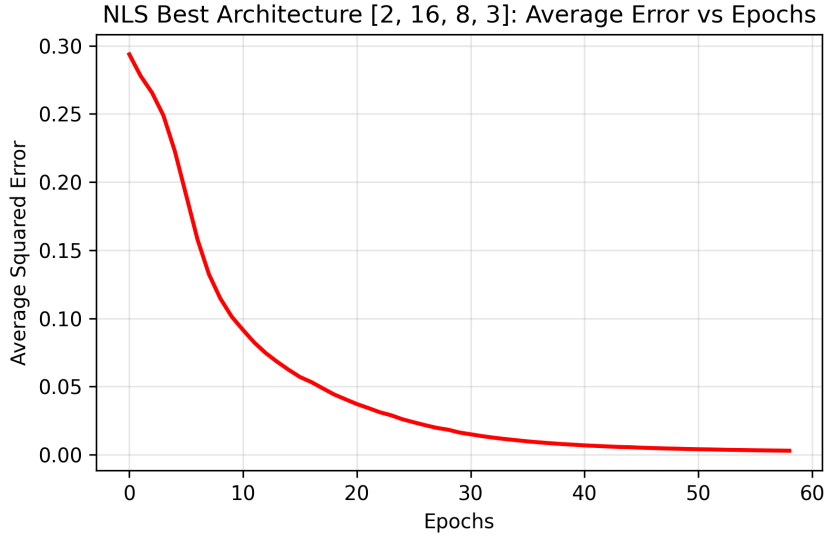


Figure 6: Dataset 2 (Non-Linearly Separable): Average error vs epochs for the optimal  $[2, 16, 8, 3]$  FCNN architecture ( $\eta = 0.01$ ).

### 3.2.3 Decision Regions and Test Set Evaluation

Figure 7 demonstrates the breakthrough capability of the  $[2, 16, 8, 3]$  FCNN. In stark contrast to Assignment I (where linear perceptrons completely collapsed into predicting the majority outer ring), the two-hidden-layer network constructs **closed non-linear concentric contours**. The network encloses the central cluster (Class 0, dark blue), wraps an isolating envelope around the intermediate crescent (Class 1, white), and partitions the outer ring (Class 2, red) with zero classification error.

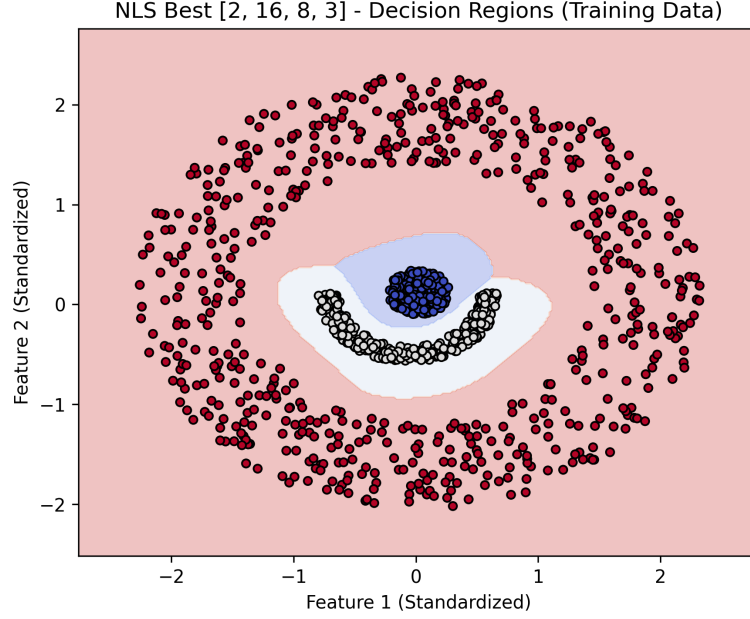


Figure 7: Dataset 2 (NLS): Non-linear closed decision regions for optimal [2, 16, 8, 3] FCNN superimposed on standardized training exemplars.

Table 5 presents the test metrics on  $N_{te} = 360$  unseen samples. The test confusion matrix confirms flawless classification:

$$\mathbf{C}_{\text{test}} = \begin{bmatrix} 60 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 200 \end{bmatrix} \quad (18)$$

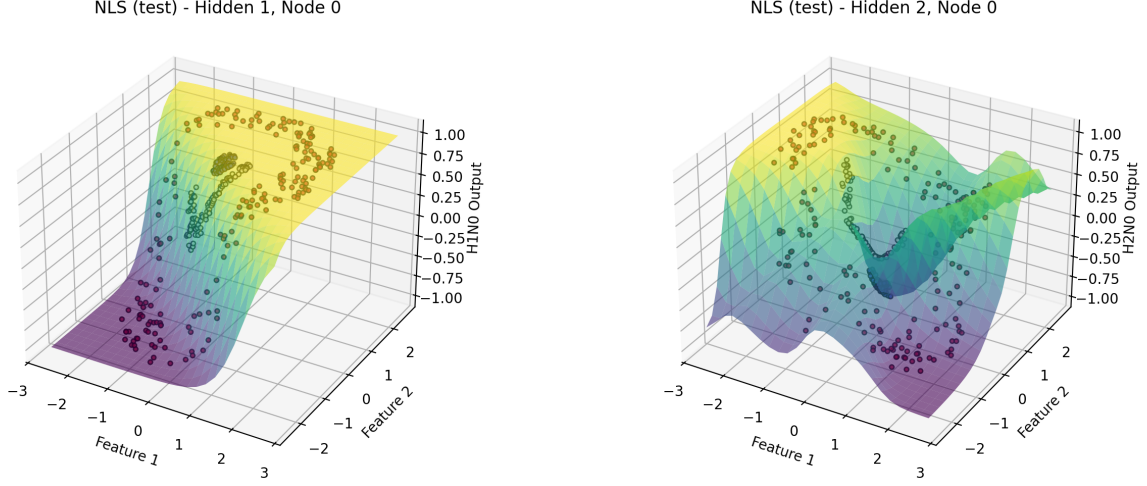
Table 5: Dataset 2 (NLS): Class-Wise Test Set Evaluation for Optimal [2, 16, 8, 3] FCNN

| Class Label          | Precision   | Recall      | F1-Score    | Overall Metrics                                                                          |
|----------------------|-------------|-------------|-------------|------------------------------------------------------------------------------------------|
| Class 0              | 1.00        | 1.00        | 1.00        | <b>Test Accuracy: 100.00%</b><br><b>Mean Precision: 1.00</b><br><b>Mean Recall: 1.00</b> |
| Class 1              | 1.00        | 1.00        | 1.00        |                                                                                          |
| Class 2              | 1.00        | 1.00        | 1.00        |                                                                                          |
| <b>Macro Average</b> | <b>1.00</b> | <b>1.00</b> | <b>1.00</b> | <b>Mean F1-Score: 1.00</b>                                                               |

### 3.2.4 Hidden Node Activation Surface Analysis Across Layers

Figures 8a and 8b illustrate representative hidden unit activations across Hidden Layer 1 (NLS\_test\_H1\_N0.png) and Hidden Layer 2 (NLS\_test\_H2\_N0.png) evaluated on the test set.

- **Layer 1 Representation:** Node 0 forms an oriented planar sigmoidal ridge across the 2D plane, slicing through a portion of the concentric geometry.
- **Layer 2 Representation:** Node 0 non-linearly synthesizes the multiple directional ridges generated by Layer 1. This forms a complex, localized activation well that curves around the central boundary, enabling the network to resolve concentric topology.



(a) Hidden Layer 1, Node 0 (NLS\_test\_H1\_N0.png) (b) Hidden Layer 2, Node 0 (NLS\_test\_H2\_N0.png)

Figure 8: Dataset 2 (NLS): 3D activation surfaces of representative hidden nodes on the test set.

### 3.3 Comparison with Assignment 1 Single-Neuron Model Baseline

Table 6 provides a direct, rigorous comparison between the single-neuron OVO baseline from Assignment I and the optimal FCNN architectures evaluated under identical 60%–20%–20% data partitioning.

Table 6: Classification Benchmarking: Single-Neuron OVO Baseline vs Optimal Multilayer FCNN

| Dataset                      | Single-Neuron<br>OVO Acc (%) | Best FCNN<br>Architecture | FCNN Test<br>Acc (%) | Absolute<br>Gain |
|------------------------------|------------------------------|---------------------------|----------------------|------------------|
| Linearly Separable (LS)      | 99.33%                       | [2, 4, 3]                 | <b>99.67%</b>        | +0.34%           |
| Non-Linearly Separable (NLS) | 55.56%                       | [2, 16, 8, 3]             | <b>100.00%</b>       | <b>+44.44%</b>   |

### 3.4 Inferences on Classification Experiments

- Overcoming Geometric Linearity:** On Dataset 1 (LS), both models succeed (99.33% vs 99.67%), proving that when underlying data distributions are linearly separable, single-neuron models suffice. However, on Dataset 2 (NLS), the single-neuron model fails completely (55.56%, which equals the zero-rule majority class baseline  $1000/1800 \approx 55.56\%$ ). The FCNN shatters this ceiling, jumping to 100.00%.
- Mechanisms of Intermediate Hidden Representations:** The 3D hidden node activation plots in Figure 8 expose the geometric mechanism: hidden layer 1 acts as localized linear feature detectors, projecting sigmoidal ridges across  $(x_1, x_2)$ . Layer 2 computes non-linear compositions of these ridges, folding the 2D input space into an intermediate coordinate system where the concentric rings become linearly separable.
- Efficiency of Layer Depth:** The two-hidden-layer structure [2, 16, 8, 3] achieved lower loss than shallow architectures. Stacking two non-linear layers allows compositional feature reuse, requiring exponentially fewer total parameters than attempting to approximate complex circular geometries with a single massive hidden layer.

## 4 Regression Tasks: Results and Inferences

### 4.1 Dataset 1: Univariate Continuous Function Approximation

The Univariate dataset contains 1001 pairs of input  $x \in \mathbb{R}$  and continuous target  $y \in \mathbb{R}$ . As observed in Assignment I, the target forms an oscillatory non-linear sinusoidal wave.

#### 4.1.1 Architecture Exploration on Validation Data

Table 7 presents the Root Mean Squared Error (RMSE) and Percentage RMSE ( $\%RMSE = \frac{RMSE}{|\bar{y}|} \times 100$ ) across single-hidden-layer architectures with node counts  $\{2, 4, 8, 16\}$ .

Table 7: Dataset 1 (Univariate): Architecture Validation Comparison ( $N_{tr} = 600, N_{val} = 200$ )

| Architecture | Parameters | Train RMSE    | Train %RMSE  | Val RMSE      | Val %RMSE    |
|--------------|------------|---------------|--------------|---------------|--------------|
| [1, 2, 1]    | 7          | 0.1118        | 4.07%        | 0.1099        | 4.00%        |
| [1, 4, 1]    | 13         | 0.0818        | 2.98%        | 0.0748        | 2.72%        |
| [1, 8, 1]    | 25         | 0.2374        | 8.64%        | 0.2175        | 7.91%        |
| [1, 16, 1]   | 49         | <b>0.0669</b> | <b>2.43%</b> | <b>0.0637</b> | <b>2.31%</b> |

**Best Model Selection:** Architecture [1, 16, 1] achieves the lowest validation RMSE of 0.0637 ( $\%RMSE = 2.31\%$ ), demonstrating the best continuous fitting capability.

#### 4.1.2 Learning Rate Comparison and Training Dynamics

Figure 9 illustrates learning rate convergence for the [1, 16, 1] model. Unlike classification, regression with continuous linear outputs is more sensitive to SGD step size. At  $\eta = 0.08$ , early stochastic oscillations occur before stabilizing (epoch 47). At  $\eta = 0.01$ , SGD descends steadily to epoch 51, while  $\eta = 0.05$  explores further before stopping at epoch 170. Figure 10 depicts the loss curve for  $\eta = 0.01$ , falling smoothly from  $E_0 = 0.056$  to  $E_{51} = 0.0022$ .

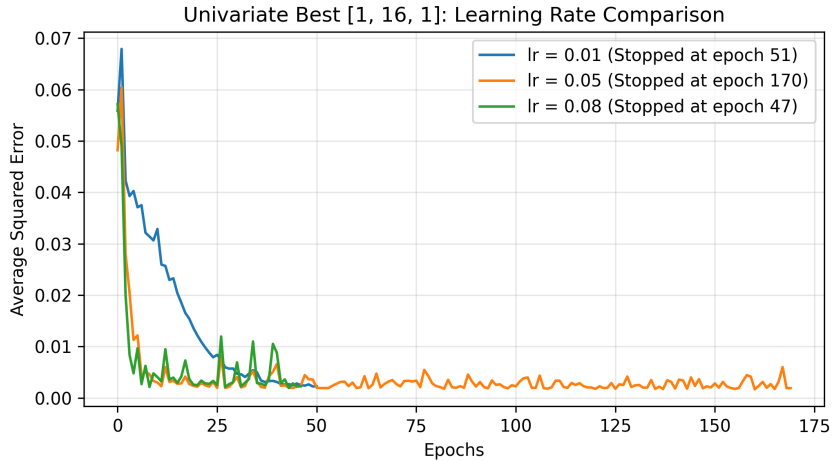


Figure 9: Dataset 1 (Univariate Regression): Average error vs epochs across learning rates  $\eta \in \{0.01, 0.05, 0.08\}$  for the [1, 16, 1] architecture.

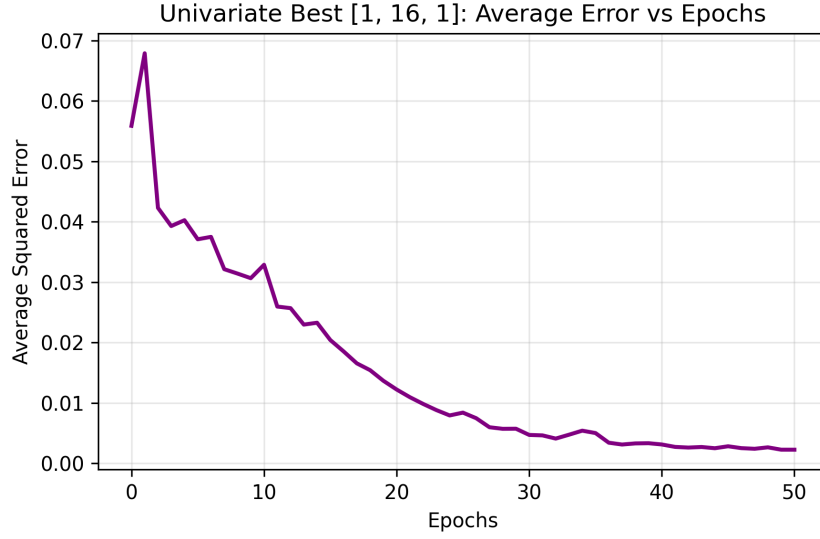


Figure 10: Dataset 1 (Univariate Regression): Average error vs epochs for the optimal  $[1, 16, 1]$  FCNN architecture ( $\eta = 0.01$ ).

#### 4.1.3 Superimposed Profiles and Parity Scatter Plots

Figures 11, 12, and 13 illustrate the model output superimposed on target data across Training, Validation, and Test partitions, respectively. In Assignment I, the linear perceptron was restricted to a single flat diagonal line. Here, the  $[1, 16, 1]$  FCNN traces the underlying sinusoidal trajectory with fidelity across all three splits.

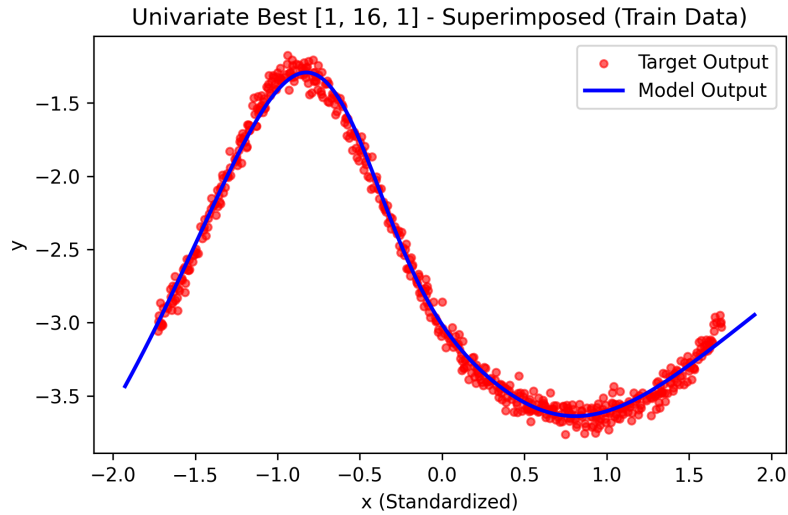


Figure 11: Dataset 1 (Univariate): Superimposed target outputs (red scatter) and  $[1, 16, 1]$  FCNN predictions (blue curve) on Training Data ( $N = 600$ ).

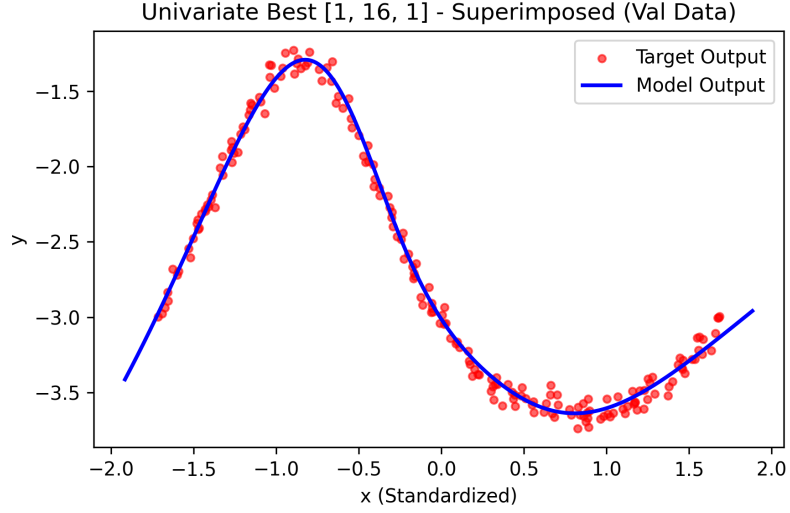


Figure 12: Dataset 1 (Univariate): Superimposed target outputs (red scatter) and [1, 16, 1] FCNN predictions (blue curve) on Validation Data ( $N = 200$ ).

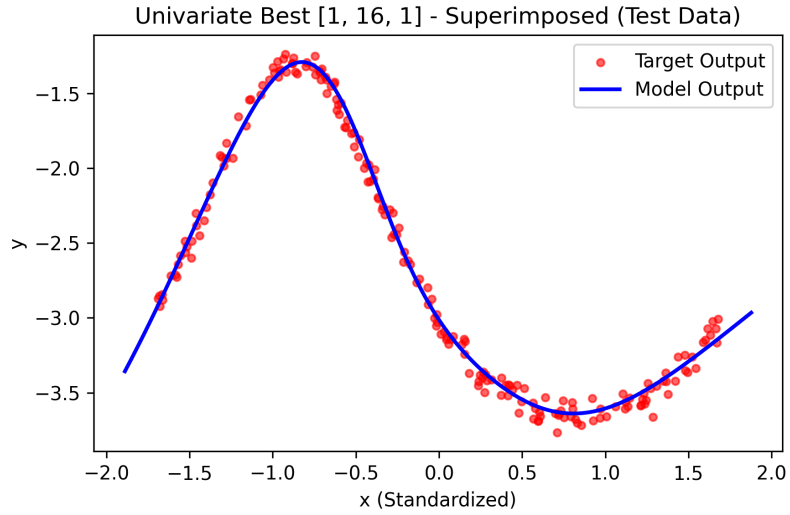


Figure 13: Dataset 1 (Univariate): Superimposed target outputs (red scatter) and [1, 16, 1] FCNN predictions (blue curve) on Test Data ( $N = 201$ ).

Figures 14, 15, and 16 present the target versus predicted parity scatter plots across the three splits. In Assignment I, the underfitted model produced an S-shaped looping curve. The FCNN collapses this distortion onto the ideal black dashed parity diagonal ( $y = \hat{y}$ ), proving high regression linearity.

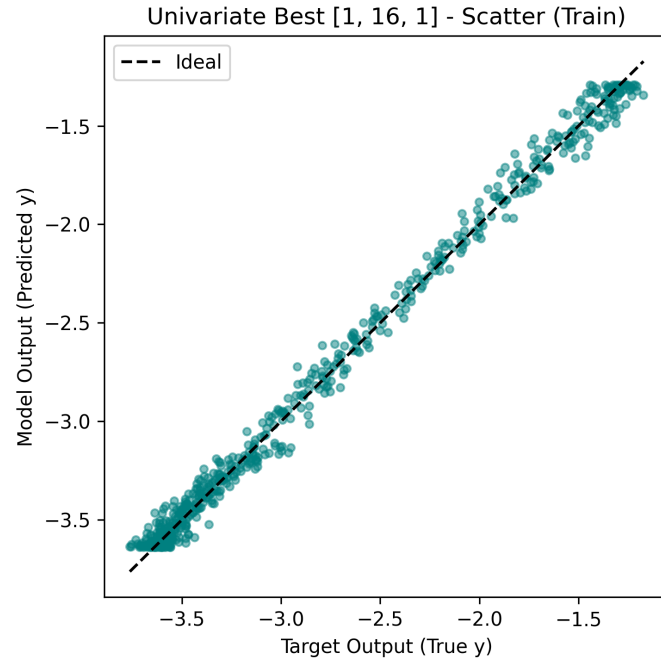


Figure 14: Dataset 1 (Univariate): Target output vs predicted model output parity scatter plot on Training Data.

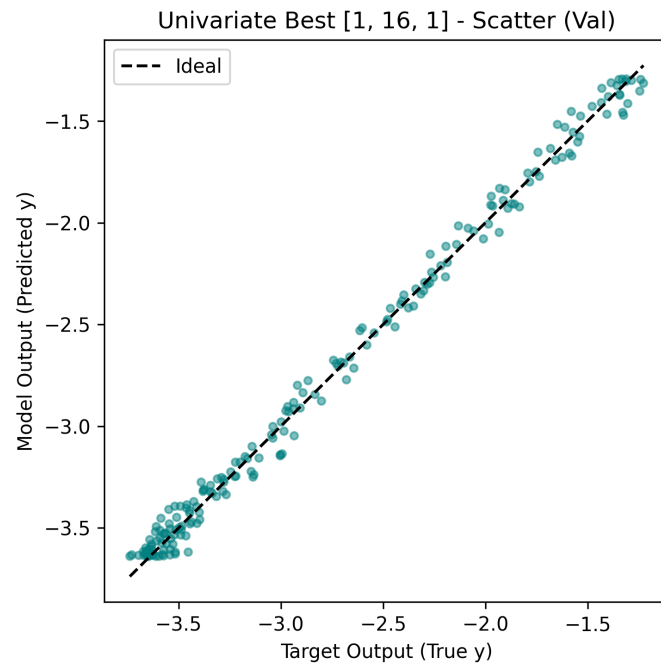


Figure 15: Dataset 1 (Univariate): Target output vs predicted model output parity scatter plot on Validation Data.



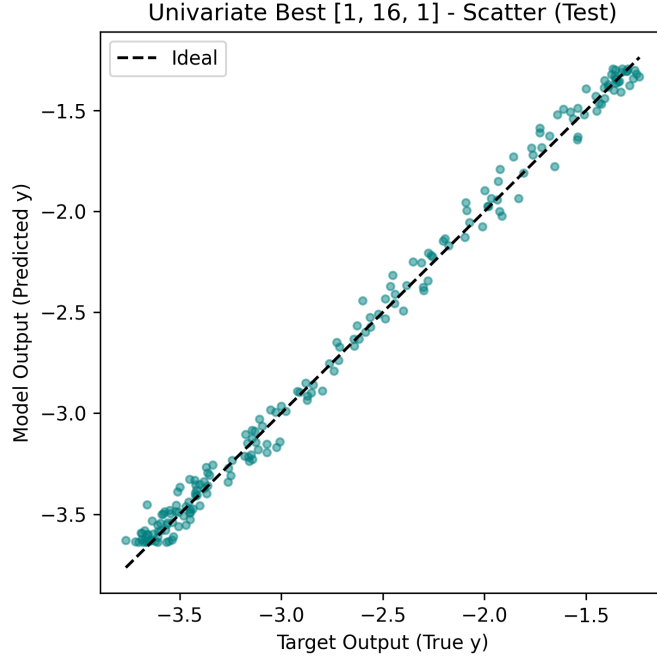


Figure 16: Dataset 1 (Univariate): Target output vs predicted model output parity scatter plot on Test Data.

#### 4.1.4 Hidden Node Basis Function Profile

Figure 17 shows the localized activation response of Hidden Layer 1 Node 0 (`Uni_Test_H1_N0.png`) across the standardized 1D test input space. The node behaves as an adaptable, localized tanh basis function that transitions sharply over a specific sub-interval of  $x$ , contributing localized curvature to the final linear summation.

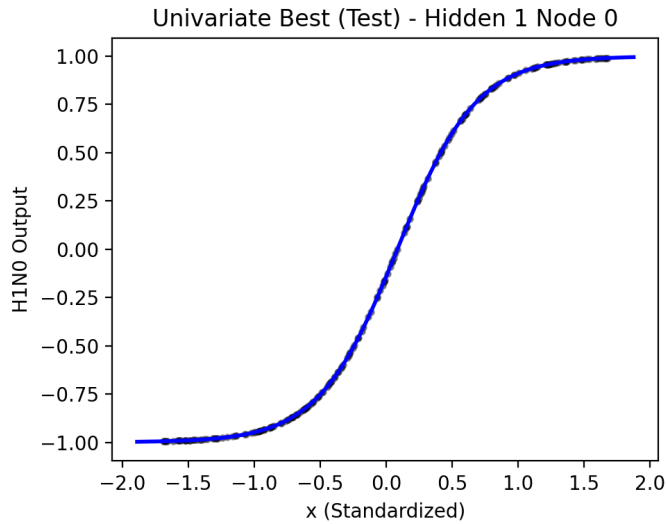


Figure 17: Dataset 1 (Univariate): 2D activation curve of representative Hidden Layer 1 Node 0 (`Uni_Test_H1_N0.png`) on the test set.

The optimal  $[1, 16, 1]$  model achieves a test RMSE of **0.0665** (%RMSE = **2.49%**) on the 201 unseen test samples.

## 4.2 Dataset 2: Bivariate Continuous Function Approximation

The Bivariate dataset comprises 10,201 exemplars with input  $(x_1, x_2) \in \mathbb{R}^2$  and continuous target  $y \in \mathbb{R}$ . As mandated, both **one-hidden-layer** and **two-hidden-layer** architectures were systematically explored.

### 4.2.1 Architecture Exploration on Validation Data

Table 8 presents the comparative evaluation across all eight candidate architectures on training ( $N_{\text{tr}} = 6120$ ) and validation ( $N_{\text{val}} = 2040$ ) partitions.

Table 8: Dataset 2 (Bivariate): Architecture Validation Comparison Across 1-Hidden and 2-Hidden Layer Configurations

| Architecture  | Hidden Layers | Parameters | Train RMSE    | Train %RMSE  | Val RMSE      | Val %RMSE    |
|---------------|---------------|------------|---------------|--------------|---------------|--------------|
| [2, 2, 1]     | 1             | 9          | 0.1708        | 2.44%        | 0.1672        | 2.40%        |
| [2, 4, 1]     | 1             | 17         | 0.0667        | 0.95%        | 0.0676        | 0.97%        |
| [2, 8, 1]     | 1             | 33         | 0.0678        | 0.97%        | 0.0696        | 1.00%        |
| [2, 16, 1]    | 1             | 65         | 0.0669        | 0.96%        | 0.0693        | 0.99%        |
| [2, 4, 4, 1]  | 2             | 37         | 0.0829        | 1.18%        | 0.0836        | 1.20%        |
| [2, 8, 4, 1]  | 2             | 65         | <b>0.0626</b> | <b>0.89%</b> | <b>0.0639</b> | <b>0.92%</b> |
| [2, 8, 8, 1]  | 2             | 105        | 0.0653        | 0.93%        | 0.0675        | 0.97%        |
| [2, 16, 8, 1] | 2             | 193        | 0.0685        | 0.98%        | 0.0705        | 1.01%        |

**Best Model Selection:** Architecture [2, 8, 4, 1] achieves the global minimum validation RMSE of **0.0639** (%RMSE = **0.92%**). Notably, it outperforms all single-hidden-layer networks, proving that a hierarchical representation (8 nodes followed by 4 nodes) captures bivariate non-linear spatial curvature better than wider single layers.

### 4.2.2 Learning Rate Dynamics and Stochastic Variance

Figure 18 presents the learning rate comparison for the [2, 8, 4, 1] architecture. In bivariate regression, stochastic gradient noise from single-sample updates is particularly prominent. At  $\eta = 0.08$ , significant stochastic oscillation occurs, requiring 153 epochs to converge. Conversely, at  $\eta = 0.01$ , the loss trajectory exhibits stable descent and terminates via early stopping at epoch 67.

This behavior occurs because a single epoch of bivariate regression executes 6,120 consecutive weight updates. Under Stochastic Gradient Descent (SGD), each individual sample provides a noisy estimate of the true gradient, where the variance of the updates scales quadratically with the learning rate:

$$\text{Variance of Weight Updates} \propto \eta^2 \quad (19)$$

Figure 19 illustrates the resulting training loss trajectory for the optimal learning rate,  $\eta = 0.01$ .

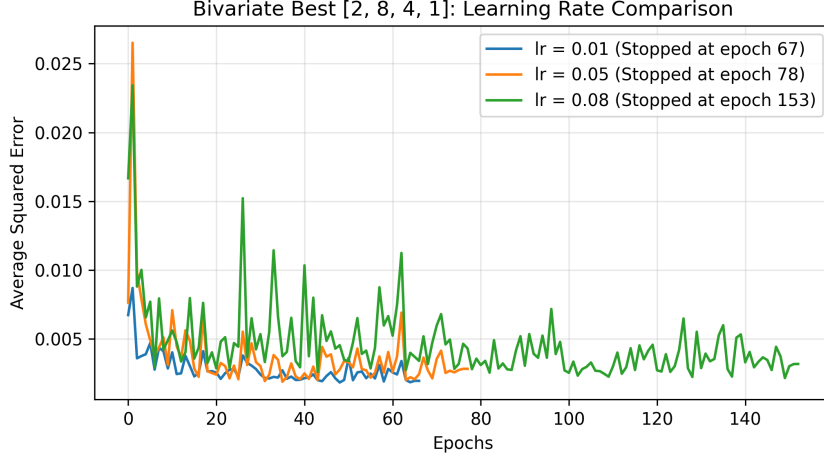


Figure 18: Dataset 2 (Bivariate Regression): Average error vs epochs across learning rates  $\eta \in \{0.01, 0.05, 0.08\}$  for the  $[2, 8, 4, 1]$  architecture.

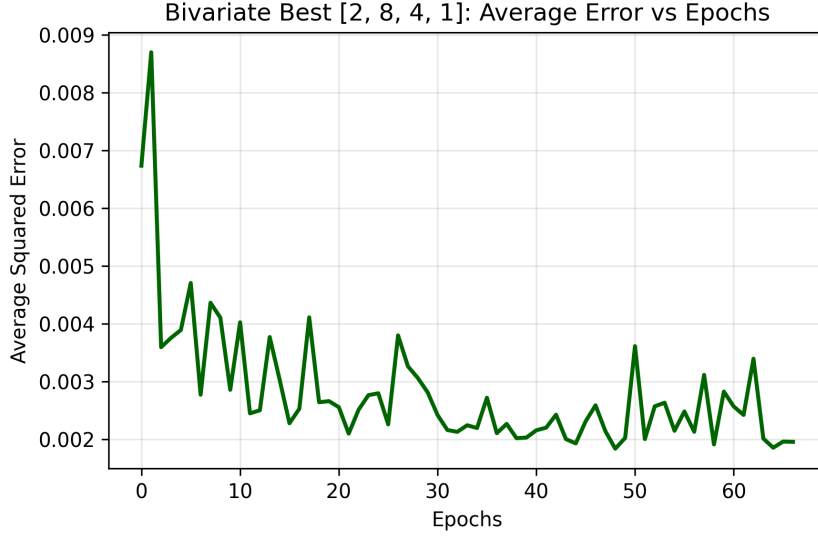


Figure 19: Dataset 2 (Bivariate Regression): Average error vs epochs for the optimal  $[2, 8, 4, 1]$  FCNN architecture ( $\eta = 0.01$ ).

#### 4.2.3 3D Surface Approximations and Parity Scatter Plots

Figures 20, 21, and 22 illustrate the 3D superimposed plots across Training, Validation, and Test partitions, respectively. The target output scatter (red spheres) is overlaid with the continuous learned manifold predicted by the  $[2, 8, 4, 1]$  FCNN (viridis mesh surface). The network captures the dual-peak ridge and saddle valleys across all three splits.

Bivariate Best [2, 8, 4, 1] - Superimposed (Train)

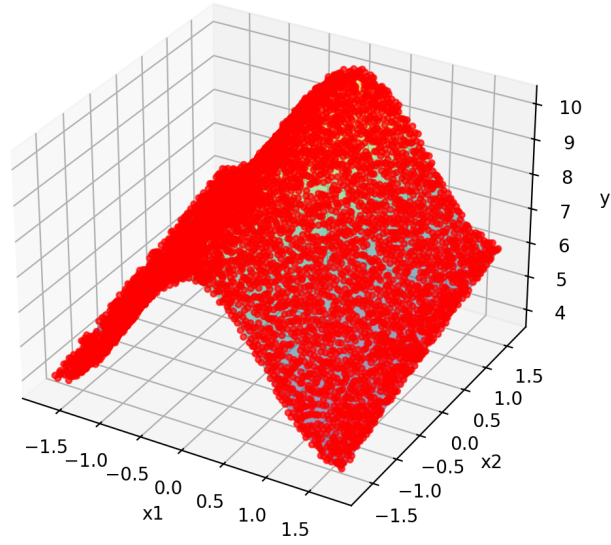


Figure 20: Dataset 2 (Bivariate): 3D superimposed target outputs and continuous manifold predicted by optimal [2, 8, 4, 1] FCNN on Training Data ( $N = 6120$ ).

Bivariate Best [2, 8, 4, 1] - Superimposed (Val)

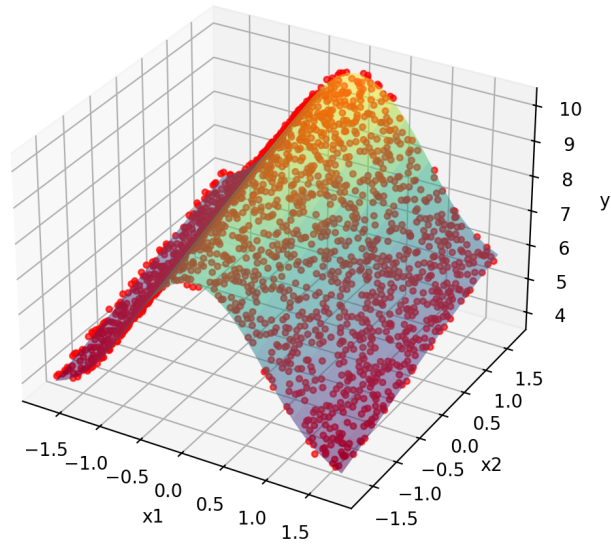


Figure 21: Dataset 2 (Bivariate): 3D superimposed target outputs and continuous manifold predicted by optimal [2, 8, 4, 1] FCNN on Validation Data ( $N = 2040$ ).

Bivariate Best [2, 8, 4, 1] - Superimposed (Test)

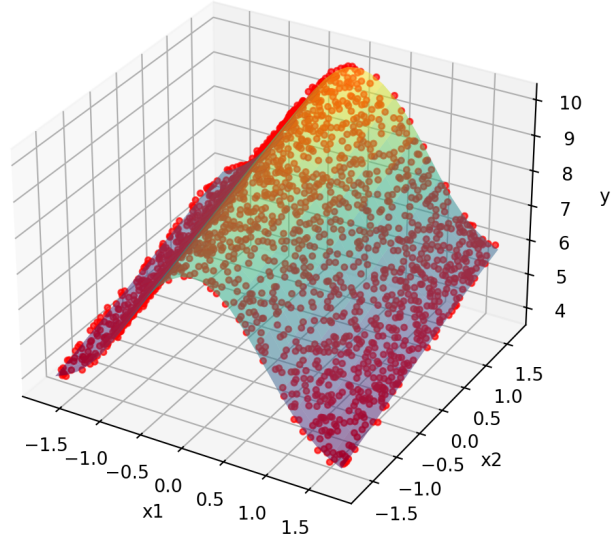


Figure 22: Dataset 2 (Bivariate): 3D superimposed target outputs and continuous manifold predicted by optimal [2, 8, 4, 1] FCNN on Test Data ( $N = 2041$ ).

Figures 23, 24, and 25 present the target versus predicted parity scatter plots across the three splits. In Assignment I, the single linear unit formed an amorphous parallelogram. The FCNN contracts all 2,041 test points into a tight, linear alignment along the ideal parity line ( $y = \hat{y}$ ).

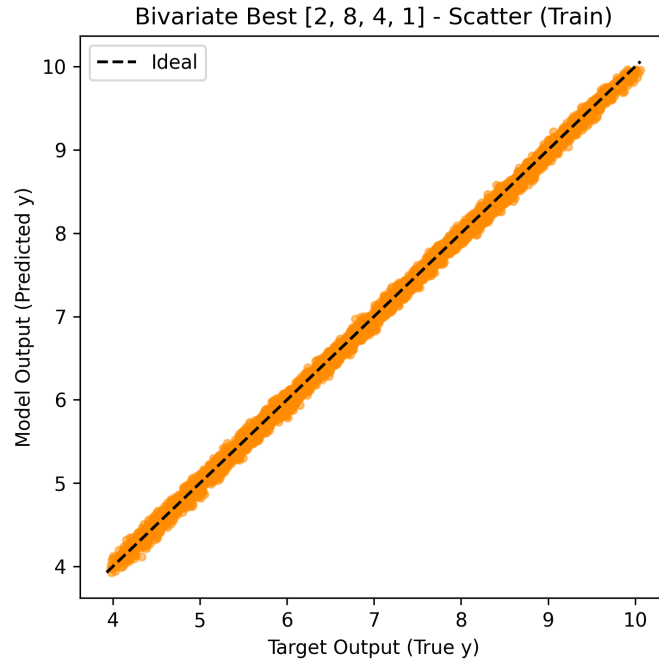


Figure 23: Dataset 2 (Bivariate): Target output vs predicted model output parity scatter plot on Training Data.

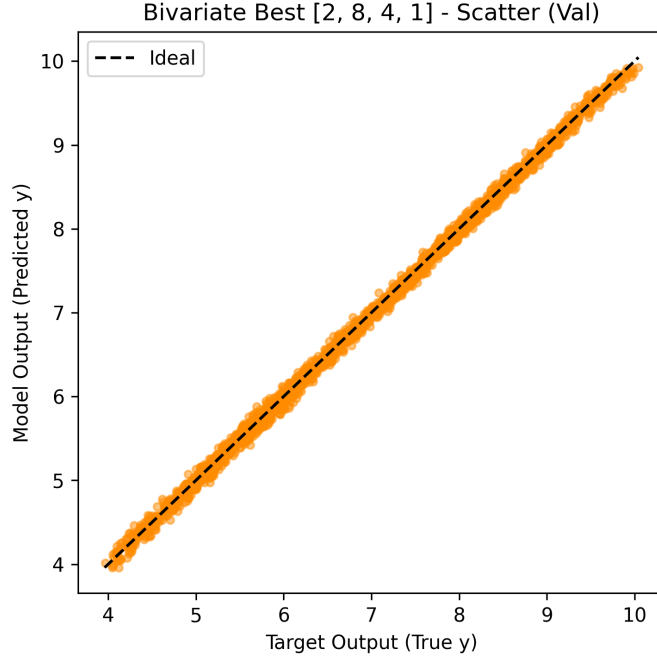


Figure 24: Dataset 2 (Bivariate): Target output vs predicted model output parity scatter plot on Validation Data.

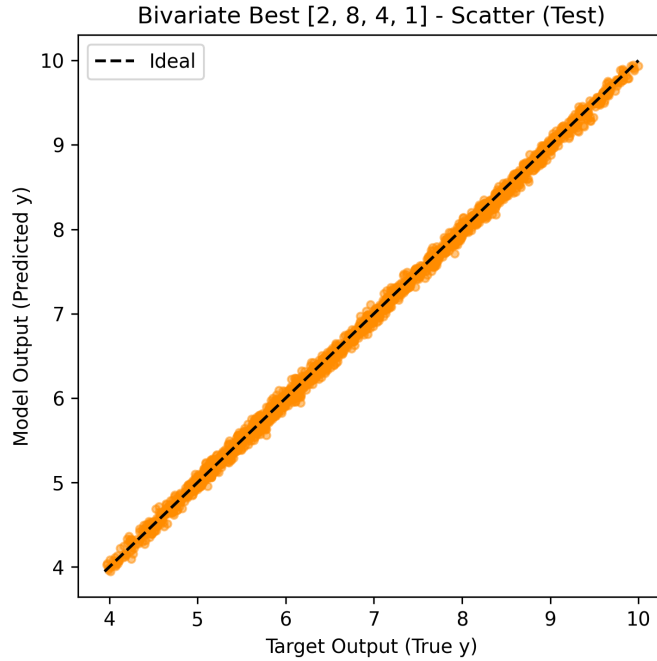


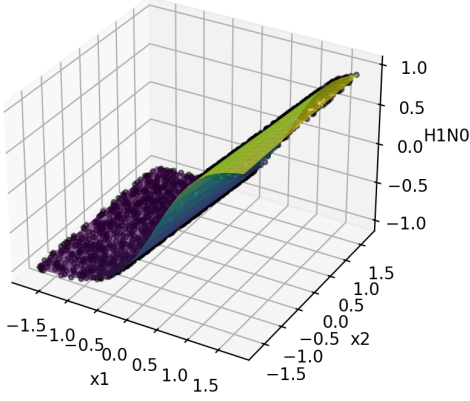
Figure 25: Dataset 2 (Bivariate): Target output vs predicted model output parity scatter plot on Test Data.

#### 4.2.4 Hidden Node Activation Surface Analysis Across Layers

Figures 26a and 26b display the 3D surface activations for Hidden Layer 1 Node 0 (Bi\_Test\_H1\_N0.png) and Hidden Layer 2 Node 0 (Bi\_Test\_H2\_N0.png) on the bivariate test split.

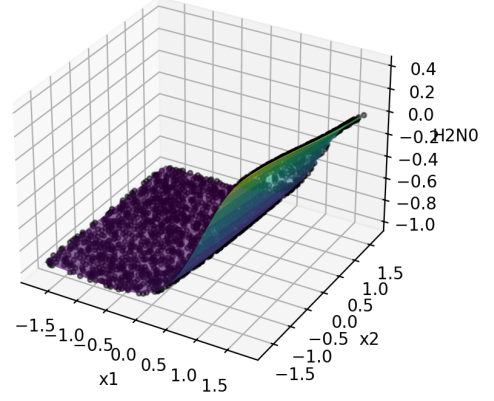
- **Layer 1 Unit:** Computes an oriented sigmoidal ramp over the  $(x_1, x_2)$  plane, acting as a flexible inclined plane.
- **Layer 2 Unit:** Combines multiple ramps from Layer 1 to produce localized saddle curvature, which the final linear output neuron integrates into the complex bivariate surface.

Bivariate Best (Test) - Hidden 1 Node 0



(a) Hidden Layer 1, Node 0 (Bi\_Test\_H1\_N0.png)

Bivariate Best (Test) - Hidden 2 Node 0



(b) Hidden Layer 2, Node 0 (Bi\_Test\_H2\_N0.png)

Figure 26: Dataset 2 (Bivariate): 3D activation surfaces of representative hidden nodes on the test set.

On the test set ( $N_{te} = 2041$ ), the optimal  $[2, 8, 4, 1]$  architecture achieves a test RMSE of **0.0625** (%RMSE = **0.90%**).

### 4.3 Comparison with Assignment 1 Single-Neuron Model Baseline

Table 9 presents the comparative evaluation between the single-neuron linear baseline and the optimal FCNN models.

Table 9: Regression Benchmarking: Single-Neuron Linear Model Baseline vs Optimal Multilayer FCNN

| Dataset         | Single-Neuron<br>Test RMSE | Best FCNN<br>Architecture | FCNN<br>Test RMSE     | Error<br>Reduction |
|-----------------|----------------------------|---------------------------|-----------------------|--------------------|
| Univariate (1D) | 0.5687 (21.29%)            | $[1, 16, 1]$              | <b>0.0665 (2.49%)</b> | <b>88.31%</b>      |
| Bivariate (2D)  | 1.4433 (20.80%)            | $[2, 8, 4, 1]$            | <b>0.0625 (0.90%)</b> | <b>95.67%</b>      |

### 4.4 Inferences on Regression Experiments

1. **Resolution of Structural Underfitting:** The single-neuron linear model from Assignment I exhibited structural underfitting ( $\sim 21\%$  RMSE) because an affine mapping  $\hat{y} = \mathbf{w}^T \mathbf{x} + b$  cannot bend. The FCNN resolves this, reducing relative error down to 0.90%.
2. **Hidden Nodes as Basis Functions:** Each hidden neuron with tanh activation acts as an adaptable, localized sigmoid ridge in the input domain:

$$h_j(\mathbf{x}) = \tanh \left( \sum_i x_i W_{ij} + b_j \right) \quad (20)$$

The output linear neuron computes an affine combination of these ridges:

$$\hat{y}(\mathbf{x}) = \sum_j h_j(\mathbf{x}) W_{j1}^{(\text{out})} + b^{(\text{out})} \quad (21)$$

As verified in the 1D and 2D hidden node activation plots in Figures 17 and 26, the network decomposes the target manifold into a set of smooth step-like basis functions whose weighted summation synthesizes the target curvature.

3. **Generalization vs Parameter Over-Parameterization:** In Table 8, increasing capacity to  $[2, 16, 8, 1]$  (193 parameters) slightly degraded validation RMSE (0.0705 vs 0.0639). This indicates that excessively wide networks begin fitting high-frequency sample noise. The  $[2, 8, 4, 1]$  network (65 parameters) achieves the optimal trade-off between bias and variance.

## 5 Overall Discussion and Key Takeaways

The empirical results across all four benchmarks validate the foundational principles of deep learning:

- **The Necessity of Depth:** While linear separability (LS) is adequately solved by a single unit, non-linear separability (NLS) and continuous function approximation demand non-linear hidden representations. Depth enables hierarchical abstraction—lower layers construct localized linear cuts, while subsequent layers synthesize enclosed convex and non-convex polytopes.
- **Optimization Dynamics in SGD:** Stochastic Gradient Descent proved effective when coupled with Xavier initialization and input standardization. While batch gradient descent (Assignment I) follows a deterministic path, online SGD exhibits stochastic fluctuations that prevent early entrapment in shallow local saddle points.
- **Learning Rate Sensitivity:** A learning rate of  $\eta = 0.01$  was universally robust, maintaining smooth convergence across all tasks. While larger rates ( $\eta = 0.08$ ) accelerated classification, they induced noise in continuous regression, establishing the necessity of empirical hyperparameter cross-validation.

## 6 Conclusion

In this programming assignment, a Fully Connected Neural Network (FCNN) was designed, implemented, and evaluated completely from first principles in Python without relying on external machine learning libraries. By deriving and implementing the backpropagation algorithm with online Stochastic Gradient Descent (SGD), the representational limitations of the single-neuron perceptron were systematically resolved.

The optimal FCNN achieved **99.67%** test accuracy on Linearly Separable classification, **100.00%** test accuracy on the Non-Linearly Separable concentric benchmark (a 44.44% leap over the single-neuron baseline), **0.0665** test RMSE on Univariate regression, and **0.0625** test RMSE on Bivariate regression (a 95.67% error reduction). These experimental investigations substantiate the Universal Approximation Theorem and demonstrate how multilayer feature hierarchies empower neural networks to model non-linear real-world phenomena.

## References

1. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
2. M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*, MIT Press, Cambridge, MA, 1969.



3. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
4. G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals, and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
5. K. Hornik, “Approximation capabilities of multilayer feedforward networks,” *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.
6. X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.
7. IIT Dharwad, *CS601T: Deep Learning Lecture Notes and Programming Guidelines*, Department of Computer Science and Engineering, 2026.